MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

Project Report

Interface Timer10 and UART2 of STM32. Write the program. To display a message on UART with a specific delay. Send ‘Welcome’ message continuously On UART with delay of 3seconds.Show proper calculations wherever required.

ARM BASED EMBEDDED SYSTEM DESIGN

Course Code: 2307220L

Second Year B.Tech. (Term-II)

2023 (NEP) Pattern

MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

Project Report

Interface Timer10 and UART2 of STM32. Write the program. To display a message on UART with a specific delay. Send ‘Welcome’ message continuously On UART with delay of 3seconds.Show proper calculations wherever required.

ARM BASED EMBEDDED SYSTEM DESIGN

Course Code: 2307220L

NAME: Om Gajanan Sapdhare

PRN: 202502070030

DIVISION: C

BATCH: C4

GROUP NO: PB10

Second Year B.Tech. (Term-II)

2023 (NEP) Pattern

COURSE NAME: ARM Based Embedded System Design

COURSE CODE: 2307220L

COURSEOBJECTIVES:

2307220L.CEO.1: To provide students thorough understanding of the ARM development tools.

2307220L.CEO.2: To develop practical understanding of ARM architecture through experimentation.

2307220L.CEO.3: To integrate hardware and software in ARM-based systems.

COURSEOUTCOMES: After successful completion of the course, students will be able to,

2307220L.CO.1: Identify and describe functions of ARM-based development tools[L2].

2307220L.CO.2: Demonstrate the ability to write and debug programs for ARM microcontrollers[L3].

2307220L.CO.3: Utilize systematic problem-solving techniques for embedded systems[L4].

Date:18/04/2026

Problem Statement No:- 10

Project Problem Statement: Interface Timer10 and UART2 of STM32. Write the program to display a message on UART with a specific delay. Send 'Welcome' message continuously on UART with delay of 3seconds.Show proper calculations wherever required.

Objective:

- To understand UART communication in STM32
- To configure and use Timer10 for delay generation
- To implement interrupt-based data transmission
- To display data on serial terminal using UART

Hardware required:

- STM32 Nucleo development board
- USB cable for programming and UART connection

Software required:

- STM32-Nucleo board
- STM32 Cube IDE
- STM32CubeMX (integrated in IDE)

Hardware Configuration:

- USART2 TX (PA2) → ST-Link Virtual COM Port
- USART2 RX (PA3) → ST-Link Virtual COM Port
- USB cable provides:
- Power supply
- Programming interface
- Serial communication

Program:

```
/* USER CODE BEGIN Header */
/**
******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
******************************************************************************
  * @attention
  *
  * Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "string.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */
```

```
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim10;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */
char msg[] = "Welcome\r\n";
int Timer_Value;
int prev = 0;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
```

```
static void MX_TIM10_Init(void);
static void MX_USART2_UART_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */
```

```
/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM10_Init();
MX_USART2_UART_Init();
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start(&htim10);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  Timer_Value = __HAL_TIM_GET_COUNTER(&htim10);

  if (Timer_Value < prev)
  {
    HAL_UART_Transmit(&huart2,
                      (uint8_t *)msg,
                      strlen(msg),
                      HAL_MAX_DELAY);
```

```
    }

    prev = Timer_Value;
  }  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();

__HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE
2);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
```

```
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM10 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM10_Init(void)
{

  /* USER CODE BEGIN TIM10_Init 0 */
```

```
  /* USER CODE END TIM10_Init 0 */

  /* USER CODE BEGIN TIM10_Init 1 */

  /* USER CODE END TIM10_Init 1 */
  htim10.Instance = TIM10;
  htim10.Init.Prescaler = 15999;
  htim10.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim10.Init.Period = 2999;
  htim10.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim10.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_Base_Init(&htim10) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM10_Init 2 */

  /* USER CODE END TIM10_Init 2 */

}

/**
  * @brief USART2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART2_UART_Init(void)
{
```

```
  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 4800;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
```

```
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */


  /* USER CODE END MX_GPIO_Init_1 */


  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();


  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);


  /*Configure GPIO pin : B1_Pin */
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);


  /*Configure GPIO pin : LD2_Pin */
  GPIO_InitStruct.Pin = LD2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
```

```
  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
```

```
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Calculation:

Formula:

$$T = \frac{(PSC+1)(ARR+1)}{f_{timer}}$$

Given:
Given

- System Clock (HSI) = **16 MHz**
- Prescaler (PSC) = **15999**
- Auto Reload Register (ARR) = **2999**

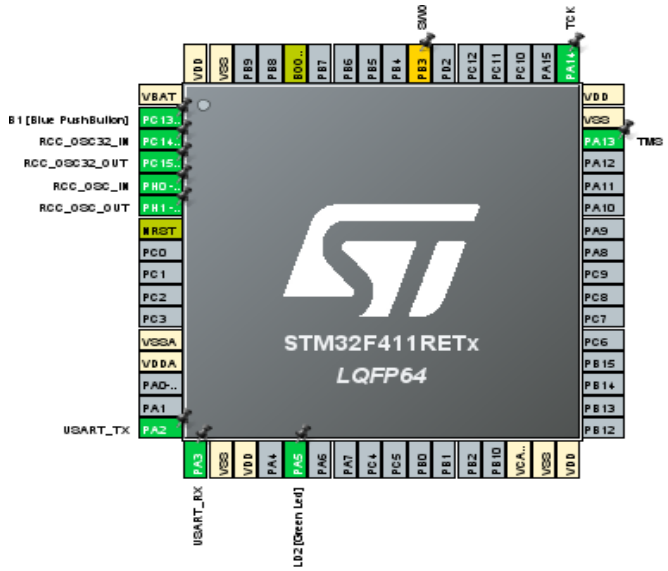
Final Result

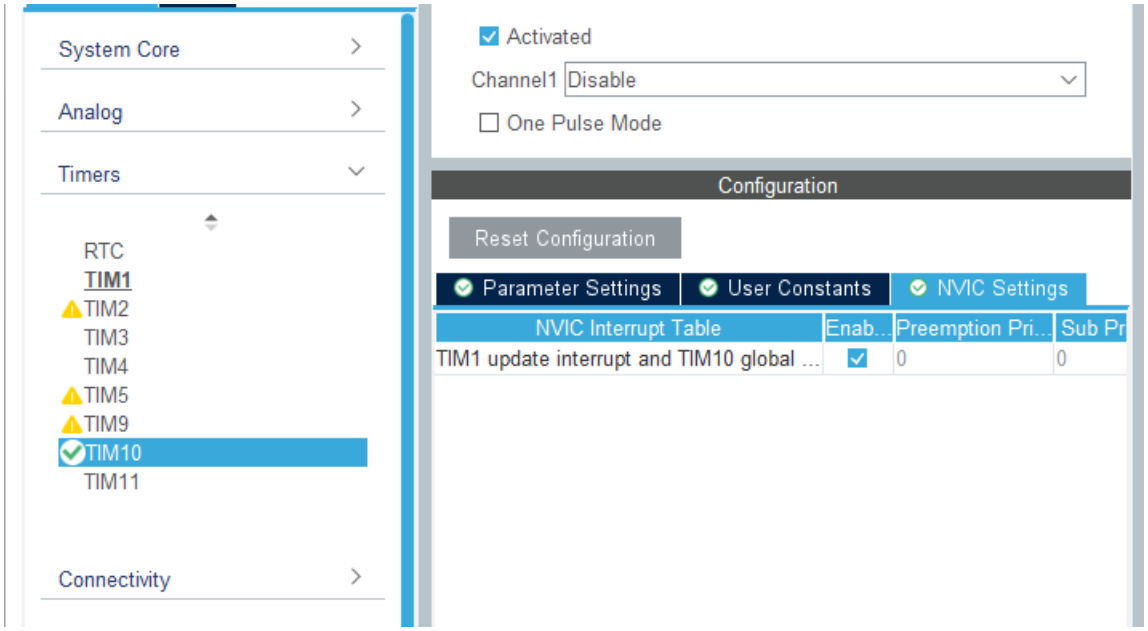
Parameter	**Value**
System Clock	16 MHz
Prescaler (PSC)	15999
ARR	2999
Timer Frequency	1000 Hz
Time per Count	1 ms
Total Counts	3000
Generated Delay	**3 Seconds**

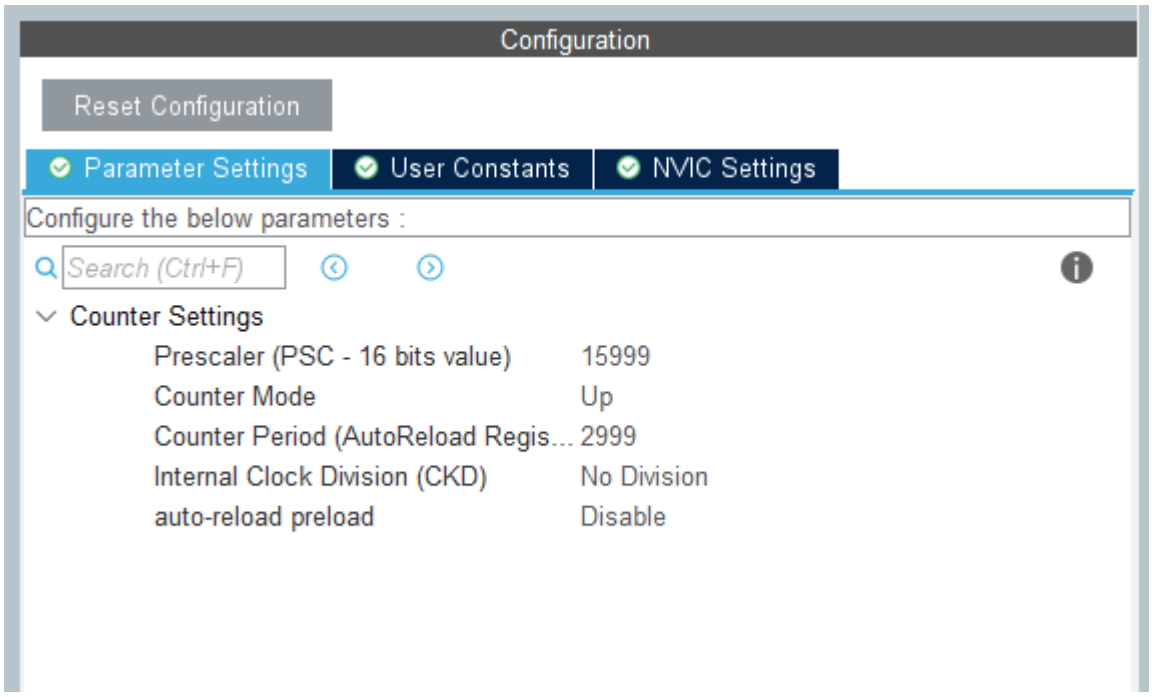
The STM32 is configured to run at **16 MHz**. By setting the **Prescaler to 15999**, the timer frequency becomes **1000 Hz**, which means each timer count represents **1 ms**. Setting the

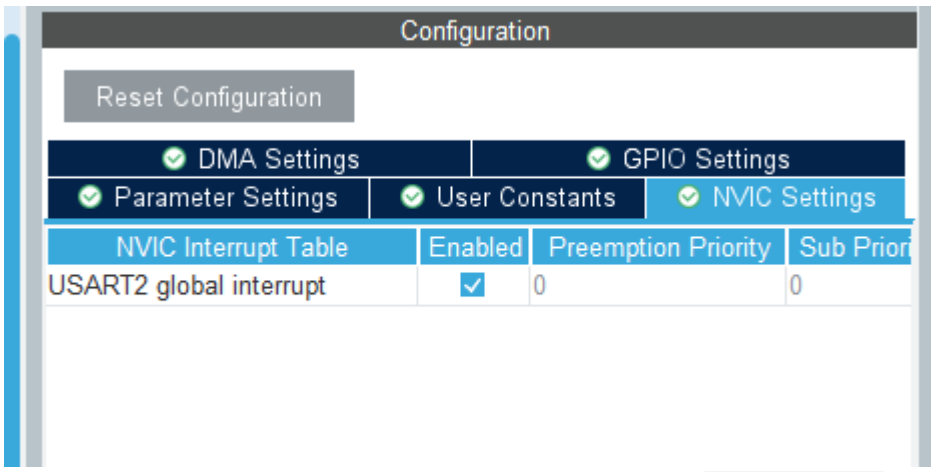
Auto Reload Register (ARR) to 2999 makes the timer count **3000 times (0–2999)**, producing a total delay of **3000 ms or 3 sec**

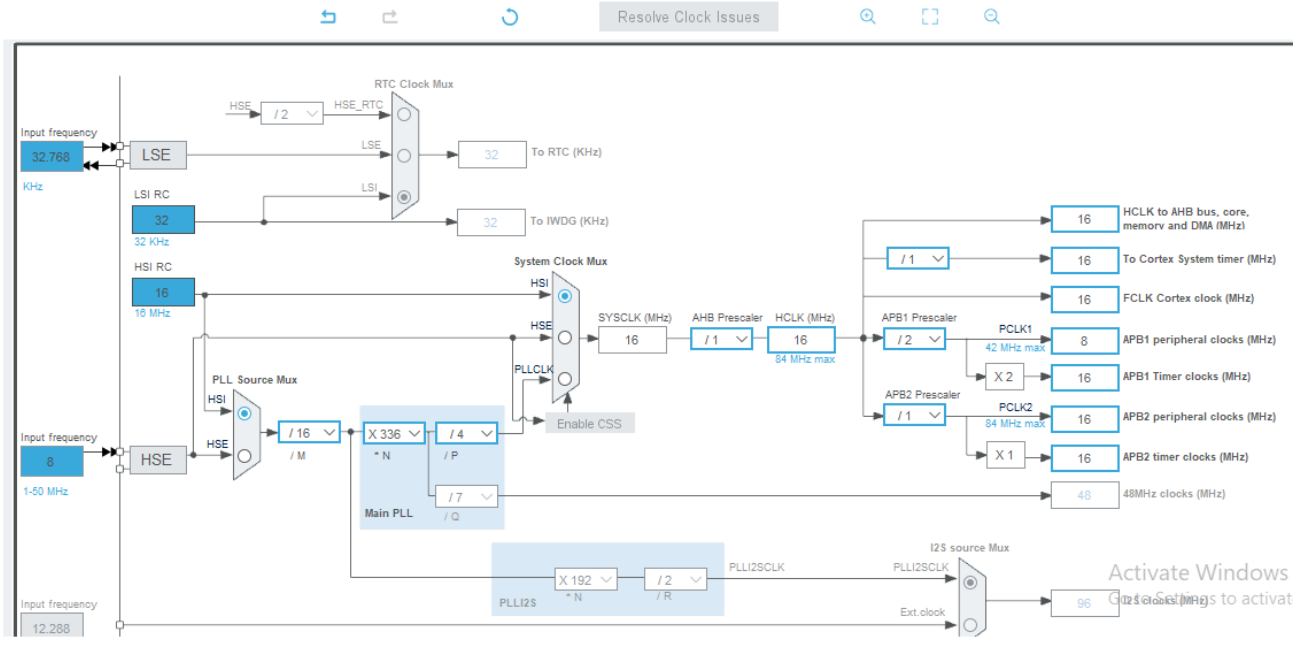
Simulation Output :-

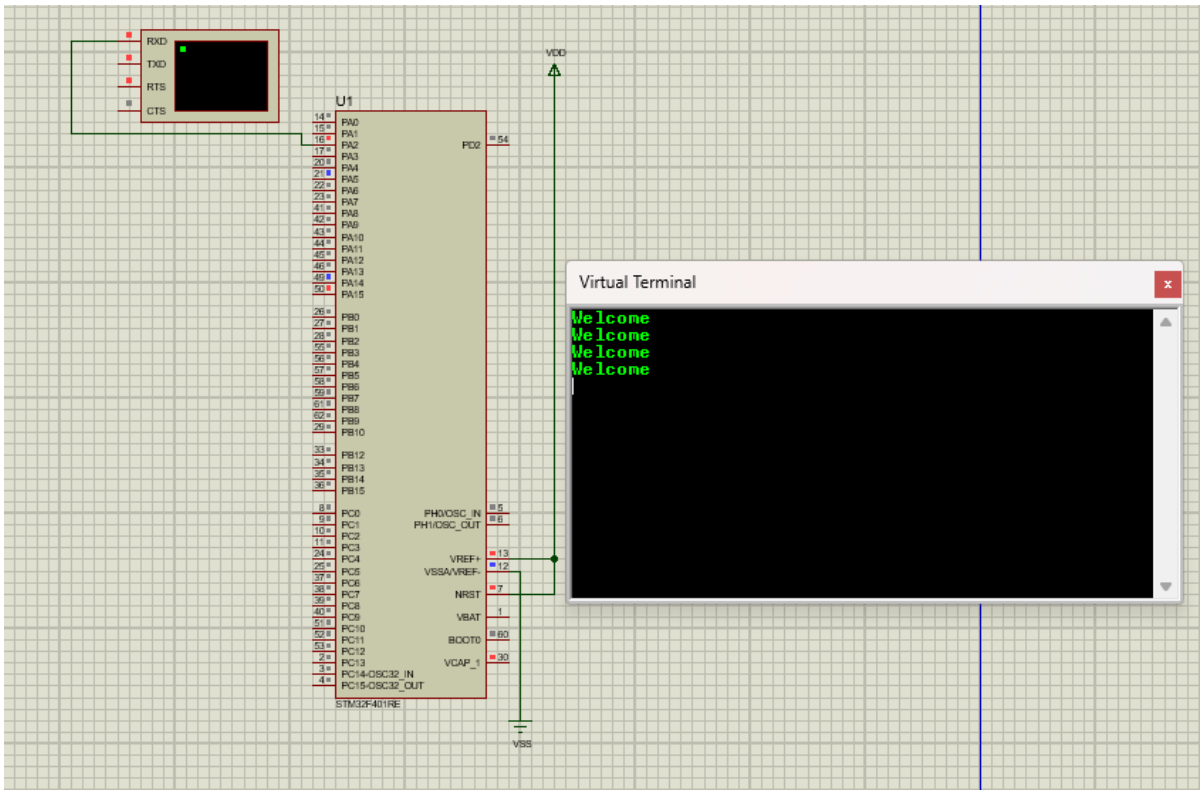




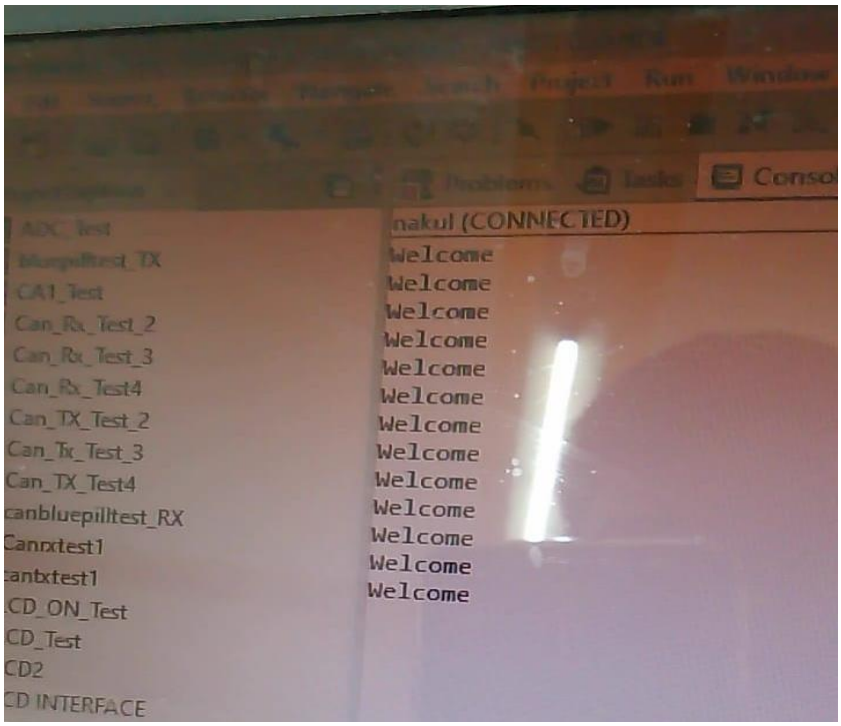

Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
Configure the below parameters :
Search (Ctrl+F)
Counter Settings
Prescaler (PSC - 16 bits value) 15999
Counter Mode Up
Counter Period (AutoReload Regis... 2999
Internal Clock Division (CKD) No Division
auto-reload preload Disable


Configuration
Reset Configuration
DMA Settings
GPIO Settings
Parameter Settings
User Constants
NVIC Settings
NVIC Interrupt Table
Enabled
Preemption Priority
Sub Prior
USART2 global interrupt
0
0


Resolve Clock Issues
RTC Clock Mux
HSE
/ 2
HSE_RTC
Input frequency
32.768
KHz
LSE
LSE
32
To RTC (KHz)
LSI RC
32
32 KHz
LSI
32
To IWDG (KHz)
HSI RC
16
16 MHz
System Clock Mux
HSI
HSE
PLLCLK
SYSCLK (MHz)
16
AHB Prescaler
/ 1
HCLK (MHz)
16
84 MHz max
APB1 Prescaler
/ 2
PCLK1
42 MHz max
APB2 Prescaler
/ 1
PCLK2
84 MHz max
16
HCLK to AHB bus, core, memory and DMA (MHz)
/ 1
16
To Cortex System timer (MHz)
16
FCLK Cortex clock (MHz)
8
APB1 peripheral clocks (MHz)
X 2
16
APB1 Timer clocks (MHz)
16
APB2 peripheral clocks (MHz)
X 1
16
APB2 timer clocks (MHz)
48
48MHz clocks (MHz)
PLL Source Mux
HSI
HSE
/ 16
/ M
X 336
* N
/ 4
/ P
/ 7
/ Q
Main PLL
Enable CSS
Input frequency
8
1-50 MHz
HSE
I2S source Mux
X 192
* N
/ 2
/ R
PLLI2S
PLLI2SCLK
Ext.clock
Input frequency
12.288
96
Activate Windows
Go to Settings to activat



Hardware Output:-



Result:

The message “Welcome” is successfully transmitted on UART every 3 seconds and displayed on the serial terminal.

Conclusion:

The project successfully demonstrates the interfacing of Timer10 and UART2 in STM32. Timer interrupt is used to generate precise delays without blocking the processor. UART communication is used to transmit data to a serial terminal. This method is efficient and suitable for real-time embedded applications.